

Lab 3 Tutorial: HLS to Overlay Workflow

Matrix Multiplication Accelerator on PYNQ-Z2

ECE 588 - Advanced Digital System Design

TA: Achyuth Gundrapally
Illinois Institute of Technology
Professor: Ken Choi

September 2026

Contents

1	Introduction	3
2	Developing HLS Code	4
2.1	C Simulation (CSIM)	4
2.2	C Synthesis (CSYN)	4
2.3	Automated Flow (Optional)	5
3	Analyzing csynth.rpt	5
4	If It Doesn't Meet Constraints	6
5	Exporting IP and Vivado Integration	6
5.1	Exporting the IP	6
5.2	Vivado Workspace Overview	7
5.3	Adding the HLS IP to Vivado	8
5.4	Verifying and Understanding the IP	10
5.5	Why IP Integration is Important	11
5.6	Vivado Block Design	12
5.7	Configuring the Processing System	12
5.8	Connecting Interfaces and Automating Design	13
6	Generating Bitstream and HWH	14
7	Understanding Overlays	17
7.1	Deploying on PYNQ-Z2	18
8	Login and Setup	18
9	Accessing the Terminal	18
10	Copying the Overlay Files	18

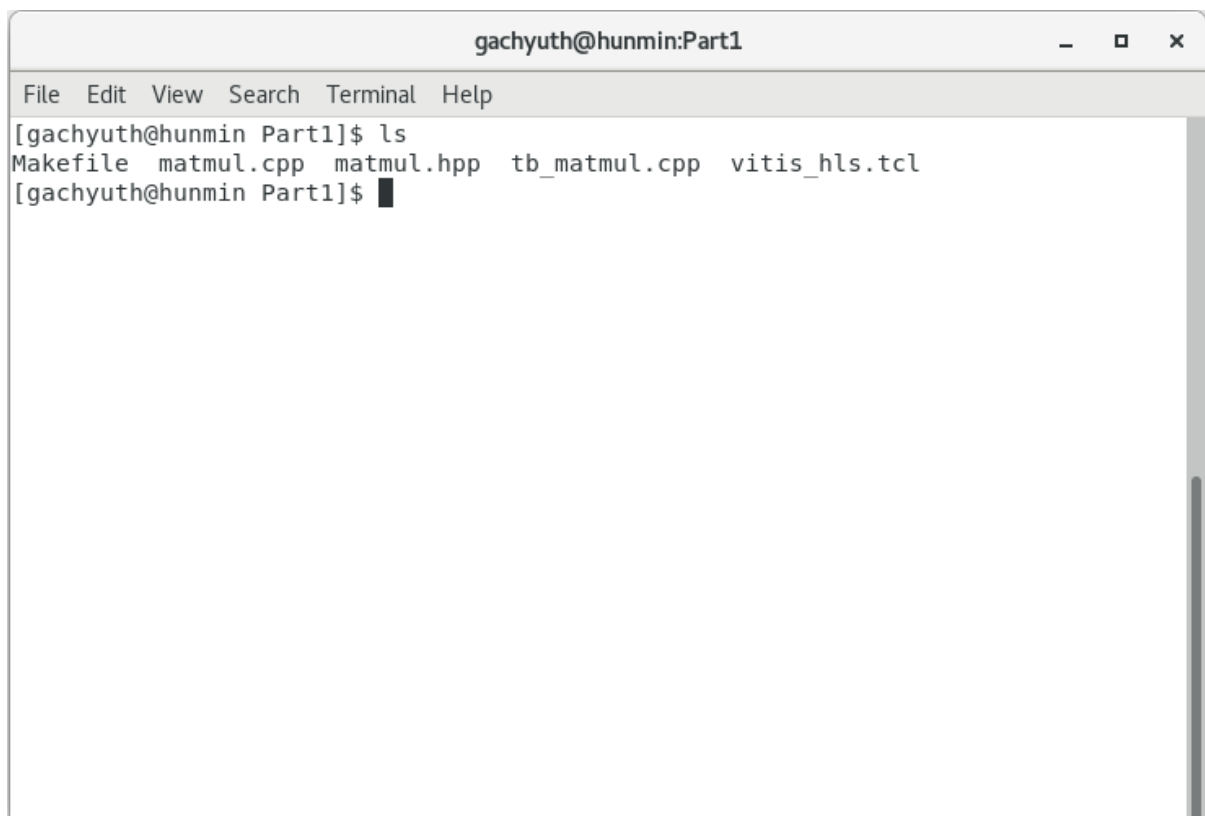
11 Running the Overlay	19
11.1 Overlay Deployment Steps	19
12 Performance Measurement Workflow	20
13 Performance Notes	21
14 Experimental Results	21
15 Measured Performance	21
16 Sample Matrix Data	22
16.1 Matrix A	22
16.2 Matrix B	22
17 Conclusion	22

1 Introduction

This tutorial provides a complete, end-to-end workflow for developing, synthesizing, and deploying a hardware accelerator on an FPGA, using matrix multiplication as an example 1. We start from high-level synthesis (HLS) in C/C++, generate synthesizable RTL, integrate the design into a Vivado project, build a bitstream and hardware hand-off files, and finally deploy the design as an **overlay** on the **PYNQ-Z2** platform. The tutorial concludes with performance measurement and CPU vs. FPGA speedup analysis in Jupyter.

The main steps are:

1. Writing and simulating HLS code
2. Running C Simulation (CSIM)
3. Running C Synthesis (CSYN)
4. Exporting IP and integrating it into Vivado
5. Generating bitstream and hardware handoff files
6. Deploying the overlay and running matrix multiplication on the PYNQ-Z2 board



```
gachyuth@hunmin:Part1
File Edit View Search Terminal Help
[gachyuth@hunmin Part1]$ ls
Makefile  matmul.cpp  matmul.hpp  tb_matmul.cpp  vitis_hls.tcl
[gachyuth@hunmin Part1]$
```

Figure 1: Matmul Files

2 Developing HLS Code

High-Level Synthesis (HLS) allows designers to describe hardware functionality using high-level C/C++ code and convert it into synthesizable RTL. This enables rapid hardware design without the need to write Verilog/VHDL manually.

```
[gachyuth@hunmin Part1]$ source /import/scripts/ece588.cshrc

Environment setting complete for 2019.2 & 2022.2 Xilinx Vivado, Vitis, Vitis_HLS

To execute Vivado2022, type 'vivado'
To execute Vivado2019, type 'vivado2019'
To execute Vitis, type 'vitis'
To execute Vitis HLS, type 'vitis_hls'

Environment setting complete for Modelsim-Intel FPGA Starter Edition Ver. 20.1

To execute Modelsim-Intel, type 'vsim'

Environment setting complete for Virtuoso

To execute Virtuoso, type 'virtuoso'
[gachyuth@hunmin Part1]$
```

Figure 2: Source Import Scripts

2.1 C Simulation (CSIM)

CSIM is a functional simulation of your C/C++ design with its testbench.

- Validates algorithm correctness against golden reference data.
- *Does not* reflect hardware timing or resource usage.

Important: Always ensure CSIM passes before proceeding. Failing CSIM means functional errors exist in the design.

2.2 C Synthesis (CSYN)

CSYN compiles your C/C++ into RTL and provides reports with:

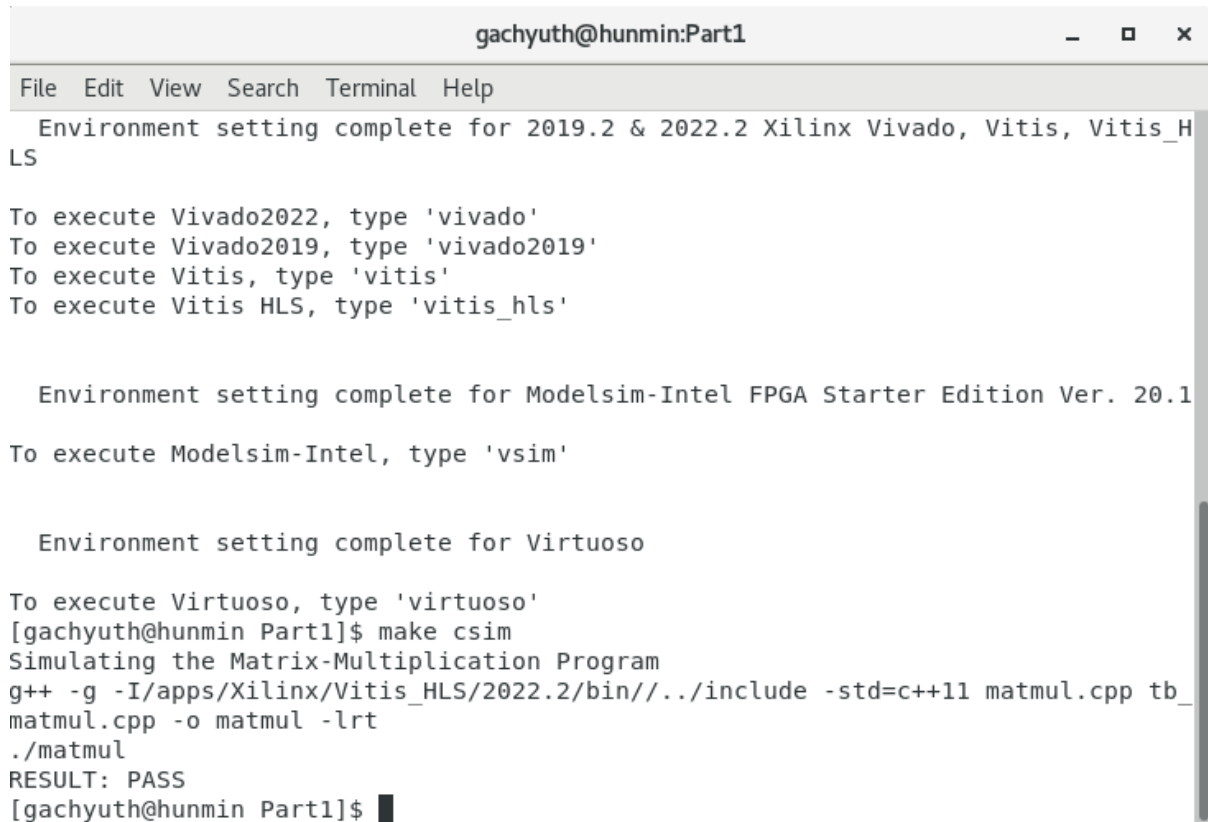
- **Resource utilization:** LUTs, FFs, BRAMs, DSPs.
- **Performance:** Latency, initiation interval (II), and estimated Fmax.

Tip: Check the synthesis report to ensure timing and resource goals for PYNQ-Z2 (XC7Z020) are met.

2.3 Automated Flow (Optional)

If provided, a Makefile and vitis_hls.tcl can automate the flow:

- `make csim` – run C simulation 3
- `make csyn` – run synthesis & export IP for Vivado 4



```
gachyuth@hunmin:Part1
File Edit View Search Terminal Help
Environment setting complete for 2019.2 & 2022.2 Xilinx Vivado, Vitis, Vitis_HLS
To execute Vivado2022, type 'vivado'
To execute Vivado2019, type 'vivado2019'
To execute Vitis, type 'vitis'
To execute Vitis HLS, type 'vitis_hls'

Environment setting complete for Modelsim-Intel FPGA Starter Edition Ver. 20.1
To execute Modelsim-Intel, type 'vsim'

Environment setting complete for Virtuoso
To execute Virtuoso, type 'virtuoso'
[gachyuth@hunmin Part1]$ make csim
Simulating the Matrix-Multiplication Program
g++ -g -I/apps/Xilinx/Vitis_HLS/2022.2/bin/./include -std=c++11 matmul.cpp tb_
matmul.cpp -o matmul -lrt
./matmul
RESULT: PASS
[gachyuth@hunmin Part1]$
```

Figure 3: Running make csim

3 Analyzing csynth.rpt

After synthesis, review the `csynth.rpt` report carefully:

- **Latency:** Min/max cycles required per operation.
- **Initiation Interval (II):** Aim for **1** (fully pipelined).
- **Clock period:** Target **10 ns (100 MHz)**.
- **Resource usage:** Ensure LUT, FF, DSP usage fits device constraints.
- **Interfaces:** Confirm correct AXI4-Lite control and data interfaces.

```
gachyuth@hunmin:Part1
File Edit View Search Terminal Help
INFO: [RTMG 210-278] Implementing memory 'matmul_Matrix_B_BRAM_V_RAM_AUTO_1R1W' using auto RAMs.
INFO: [RTMG 210-278] Implementing memory 'matmul_Matrix_C_BRAM_V_RAM_AUTO_1R1W' using auto RAMs.
INFO: [HLS 200-111] Finished Creating RTL model: CPU user time: 0.83 seconds. CPU system time: 0.02 seconds. Elapsed time: 0.85 seconds; current allocated memory: 589.660 MB.
INFO: [HLS 200-111] Finished Generating all RTL models: CPU user time: 1.95 seconds. CPU system time: 0.09 seconds. Elapsed time: 2.05 seconds; current allocated memory: 589.660 MB.
INFO: [HLS 200-1603] Design has inferred MAXI bursts and missed bursts, see Vitis HLS GUI synthesis summary report for detailed information.
INFO: [HLS 200-111] Finished Updating report files: CPU user time: 1.31 seconds. CPU system time: 0.04 seconds. Elapsed time: 1.35 seconds; current allocated memory: 589.660 MB.
INFO: [VHDL 208-304] Generating VHDL RTL for matmul.
INFO: [VLOG 209-307] Generating Verilog RTL for matmul.
INFO: [HLS 200-790] **** Loop Constraint Status: All loop constraints were NOT satisfied.
INFO: [HLS 200-789] **** Estimated Fmax: 136.99 MHz
INFO: [HLS 200-111] Finished Command csynth_design CPU user time: 66.02 seconds. CPU system time: 2.09 seconds. Elapsed time: 68.13 seconds; current allocated memory: 320.000 MB.
INFO: [HLS 200-1510] Running: export_design -format ip_catalog
INFO: [IMPL 213-8] Exporting RTL as a Vivado IP.

***** Vivado v2022.2 (64-bit)
**** SW Build 3671981 on Fri Oct 14 04:59:54 MDT 2022
**** IP Build 3669848 on Fri Oct 14 08:30:02 MDT 2022
** Copyright 1986-2022 Xilinx, Inc. All Rights Reserved.

source run_ippack.tcl -notrace
ipx::create_core: Time (s): cpu = 00:00:06 ; elapsed = 00:00:06 . Memory (MB): peak = 1481.117 ; gain = 2.020 ; free physical = 262749 ; free virtual = 320311
INFO: [IP_Flow 19-234] Refreshing IP repositories
INFO: [IP_Flow 19-1704] No user IP repositories specified
INFO: [IP_Flow 19-2313] Loaded Vivado IP repository '/apps/Xilinx/Vivado/2022.2/data/ip'.
INFO: [Common 17-206] Exiting Vivado at Thu Sep 25 12:36:40 2025...
INFO: [HLS 200-802] Generated output file vitis.prj/solution1/impl/export.zip
INFO: [HLS 200-111] Finished Command export_design CPU user time: 16.66 seconds. CPU system time: 1.16 seconds. Elapsed time: 26.51 seconds; current allocated memory: 0.000 MB.
INFO: [HLS 200-112] Total CPU user time: 84.33 seconds. Total CPU system time: 3.6 seconds. Total elapsed time: 106.45 seconds; peak allocated memory: 589.660 MB.
INFO: [Common 17-206] Exiting vitis_hls at Thu Sep 25 12:36:51 2025...
[gachyuth@hunmin Part1]$
```

Figure 4: and exporting IP

4 If It Doesn't Meet Constraints

If the design does not meet timing or resource goals, apply HLS optimizations:

Optimization Tips

- **Pipelining:** Use `#pragma HLS PIPELINE II=1` to improve throughput.
- **Loop Unrolling:** `#pragma HLS UNROLL factor=N` to parallelize loops.
- **Array Partitioning:** Use `#pragma HLS ARRAY_PARTITION` to increase memory bandwidth.
- **Dataflow:** `#pragma HLS DATAFLOW` to overlap computation stages.
- **Bit-width Optimization:** Reduce data width if precision allows.

Iterate until latency, II, and utilization targets are satisfied.

5 Exporting IP and Vivado Integration

5.1 Exporting the IP

Once the HLS synthesis step is complete and the design meets timing and resource requirements, the next step is to export the hardware design as an IP (Intellectual Property) block. This IP block will later be integrated into a Vivado block design, where it will communicate with other components such as processors, memory controllers, and interconnects.

Follow these steps to export and prepare the IP for integration:

1. Open a terminal and launch Vivado by typing:

```
vivado &
```

2. In the Vivado welcome screen, select **Create Project**.
3. Follow the project creation wizard:

- **Project Name:** Matrix
- **Project Type:** RTL Project
- **Board Selection:** PYNQ-Z2
- Click **Next** and then **Finish** to create the project.

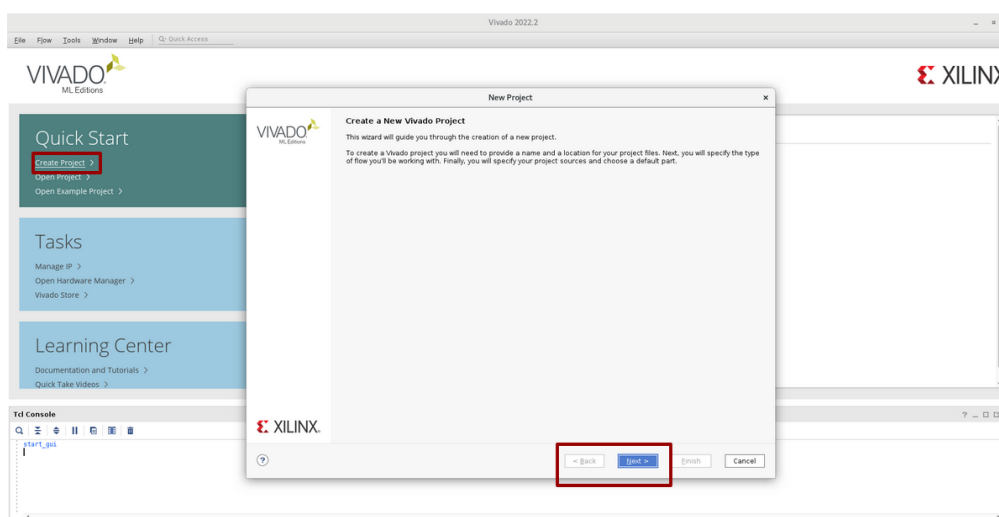


Figure 5: Creating a new Vivado project

5.2 Vivado Workspace Overview

Once the project is created, you will see the main **Vivado Design Suite** interface (Figure 10). This environment is where you will integrate your HLS IP, design the block diagram, synthesize the design, implement it, and finally generate the bitstream.

The workspace consists of several important panels:

- **Flow Navigator:** Access project creation, synthesis, implementation, and bitstream generation tasks.
- **Sources Window:** Displays design files, constraints, and generated IPs.
- **IP Catalog:** Lists all available IP blocks, including your custom HLS-generated ones.
- **Block Design Window:** Where you will visually connect and configure system components.

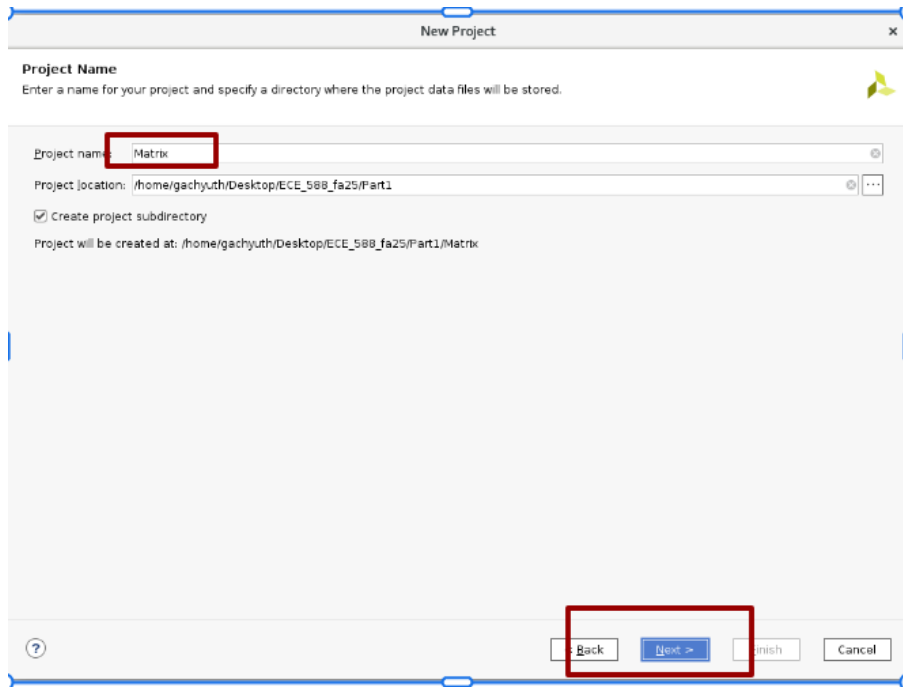


Figure 6: Naming the project

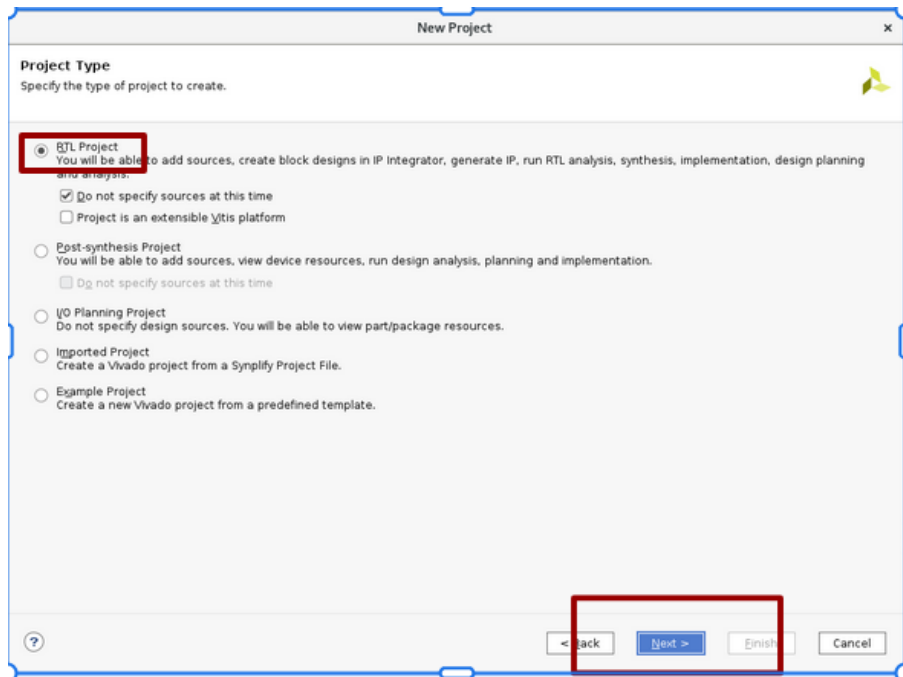


Figure 7: Selecting RTL project type

5.3 Adding the HLS IP to Vivado

With the project created, you now need to import the IP generated from Vitis HLS. This allows it to appear in the IP catalog so that it can be used like any standard Xilinx IP.

Follow these steps to add the IP:

1. In the Vivado GUI, open the **IP Catalog**.

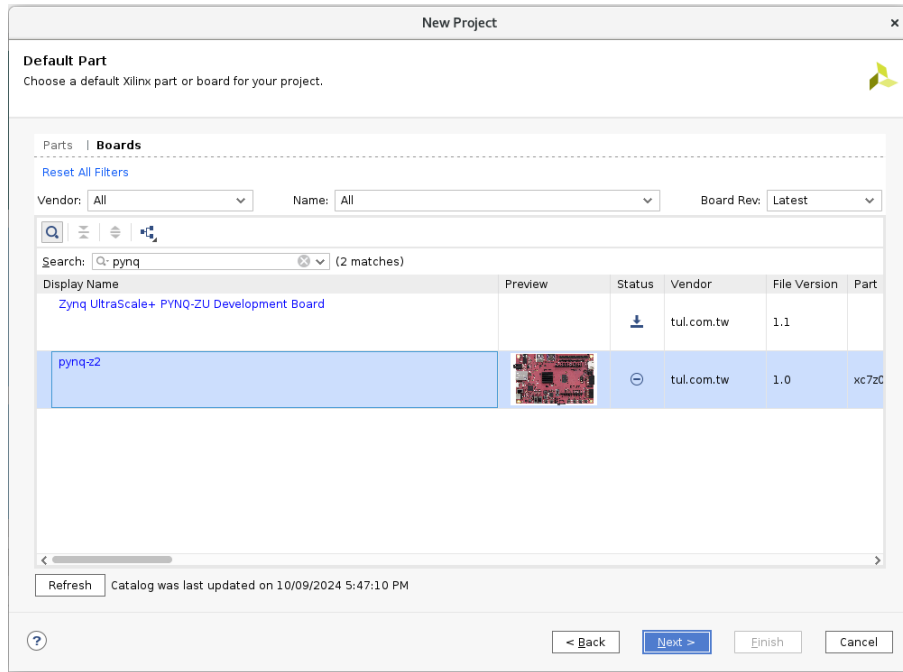


Figure 8: Choosing the PYNQ-Z2 board

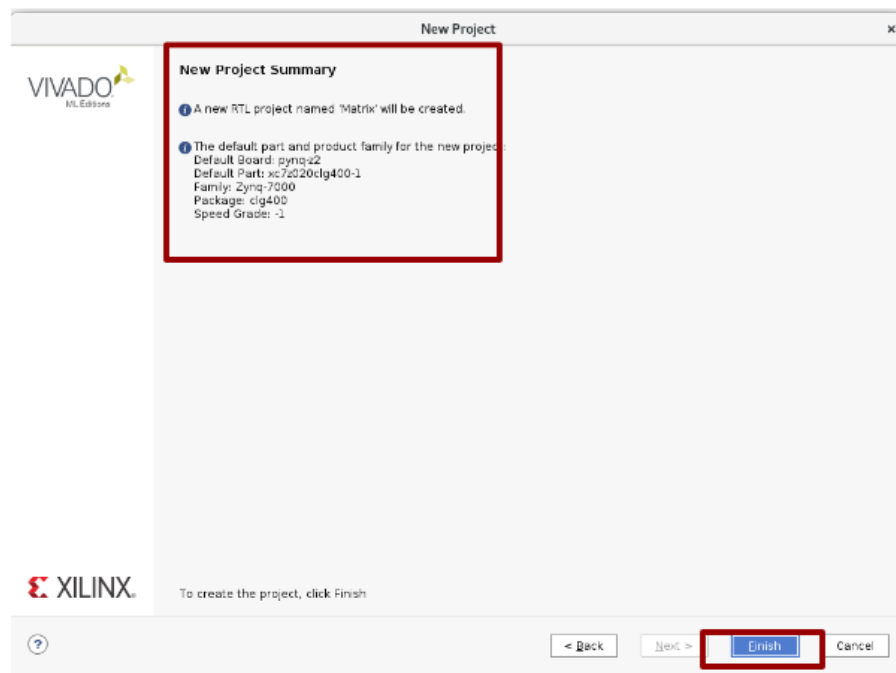


Figure 9: Project summary window

2. Right-click on **IP Repositories** and choose **Add Repository**.

When the **Add Repository** dialog appears:

1. Navigate to the directory where your HLS project was generated.
2. Select the folder inside `solution1/impl/ip` (this contains the packaged IP for the matrix multiplication accelerator).

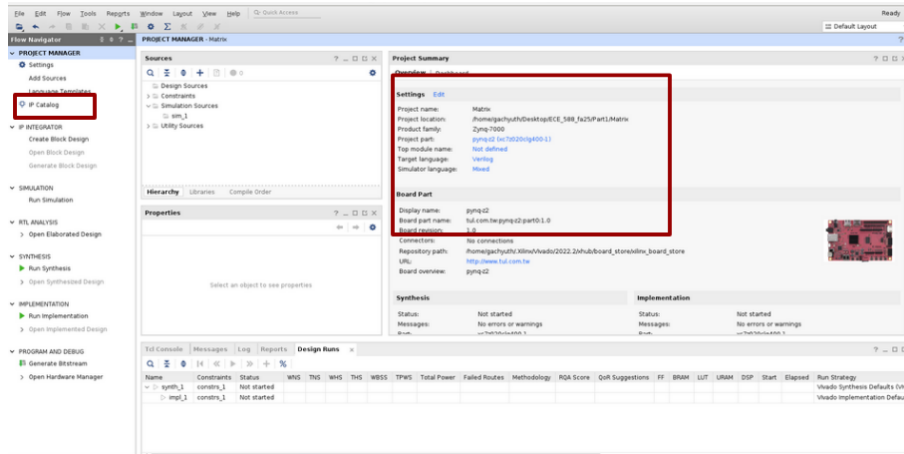


Figure 10: Vivado Design Suite workspace

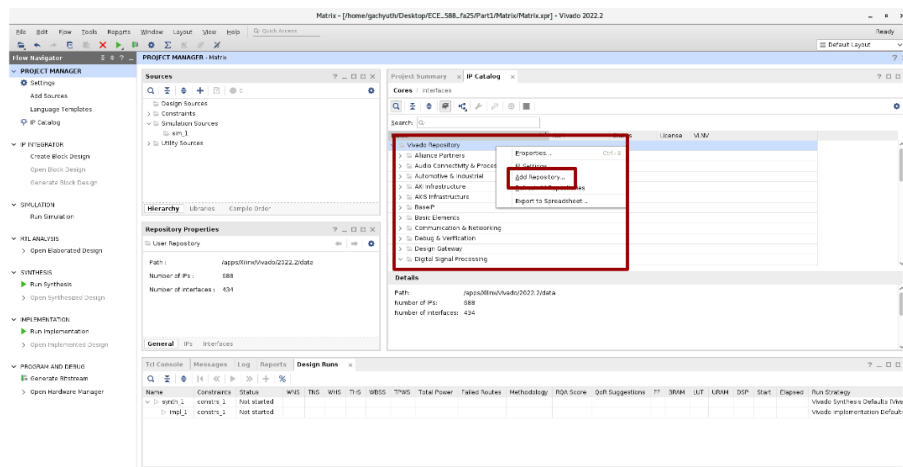


Figure 11: Adding a new IP repository

3. Click **Select Folder**.

After adding the IP, right-click on **IP Repositories** and select **Refresh**. Your new matrix multiplication IP should now be visible in the catalog and ready for use.

5.4 Verifying and Understanding the IP

Once the IP appears in the catalog, it is important to verify a few details before proceeding:

- **Name and Version:** Ensure the IP name matches your design and the version number is correct.
- **Interface Type:** Check the interfaces (e.g., AXI4-Lite for control, AXI4-Stream or AXI4-MM for data) match the design specification.
- **Parameters:** Review any configurable parameters or generics to ensure they are set properly.

Tip: If the IP does not appear, repackage it from Vitis HLS and ensure the repository path is correct.

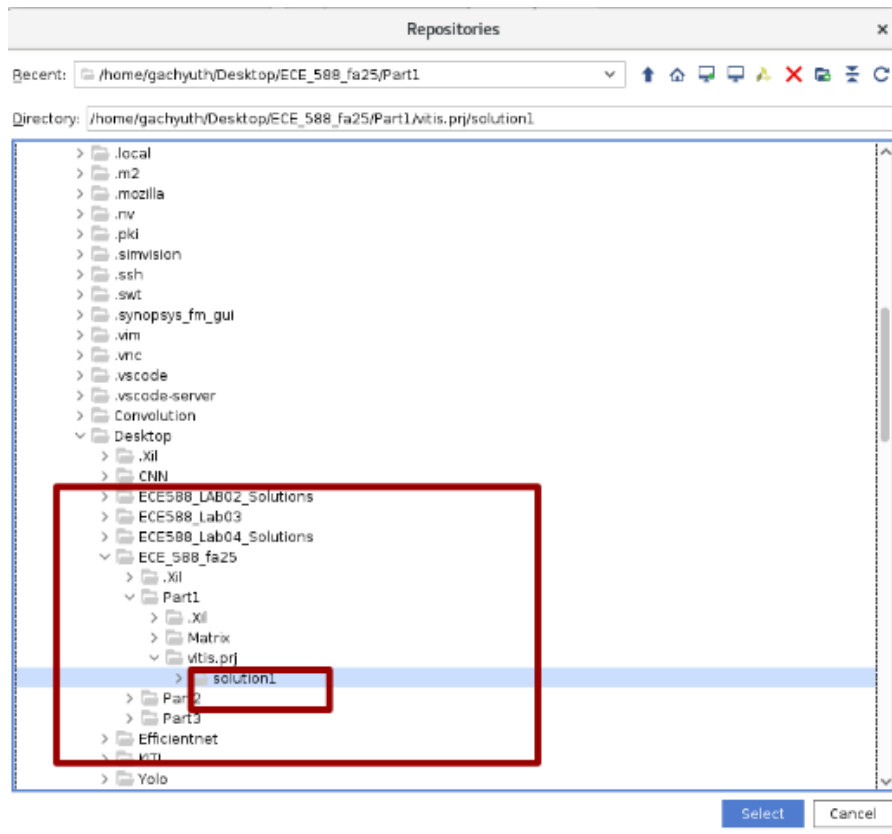


Figure 12: Selecting the IP solution directory

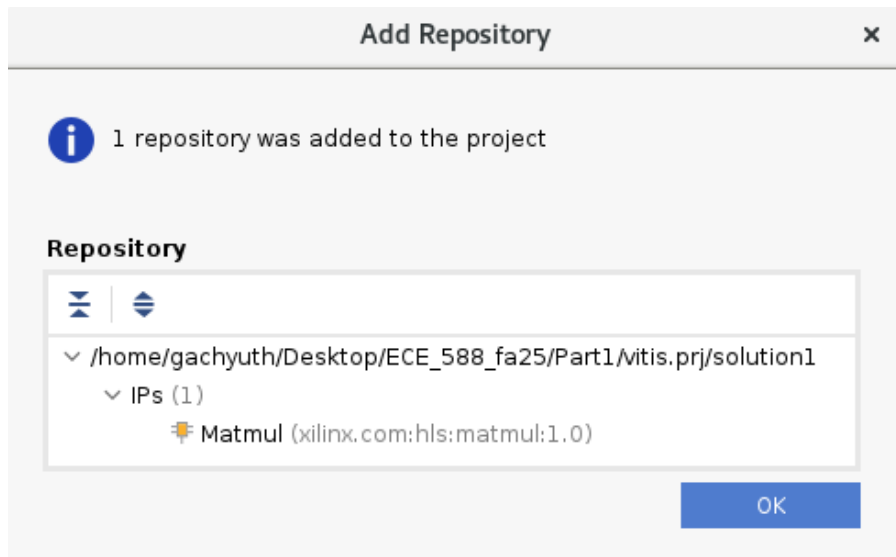


Figure 13: IP successfully added to the repository

5.5 Why IP Integration is Important

Adding the HLS IP to Vivado is a critical step because it bridges the gap between your C/C++ algorithm and the hardware system. Once integrated:

- Your algorithm can interact with the Zynq Processing System (PS) via standard AXI interfaces.

- It can communicate with memory, DMA engines, and other peripherals in the system.
- You can reuse and parameterize the IP for future projects without re-synthesizing.

This modular approach enables a structured hardware design flow and significantly accelerates development time for complex FPGA-based systems.

5.6 Vivado Block Design

Once the IP has been added to Vivado, the next step is to build a complete hardware system by creating a block design. This system typically connects the Zynq Processing System (PS), the HLS-generated accelerator IP, and the necessary interconnect components.

1. Create a new block design from the Vivado GUI.
2. Add the following components:
 - **Zynq Processing System (PS)** – acts as the ARM Cortex-A9 CPU on the PYNQ board.
 - **Matrix Multiplication HLS IP** – your custom accelerator block.
 - **AXI Interconnect** and (optional) **AXI DMA** – for data movement between the PS and accelerator.
3. Connect control and data interfaces (AXI4-Lite, AXI4-MM, or AXI4-Stream).
4. Validate the design and assign address ranges.

Once Vivado is open:

- Click on **Create Block Design** and name the design **matrix**.
- Click the **+** icon and search for **matmul**, then double-click to add your accelerator IP.
- Similarly, add the **Zynq7 Processing System**.

5.7 Configuring the Processing System

Double-click on the Zynq Processing System block to open its configuration window. Then:

- Go to the **PS-PL Fabric** tab.
- Enable the **S AXI HP0** and **S AXI HP1** interfaces (these high-performance ports are used for data transfer between the PS and PL).
- Go to **Clock Configuration** and set **FCLK_CLK0** to **100 MHz**.
- Click **Apply** and then **OK**.

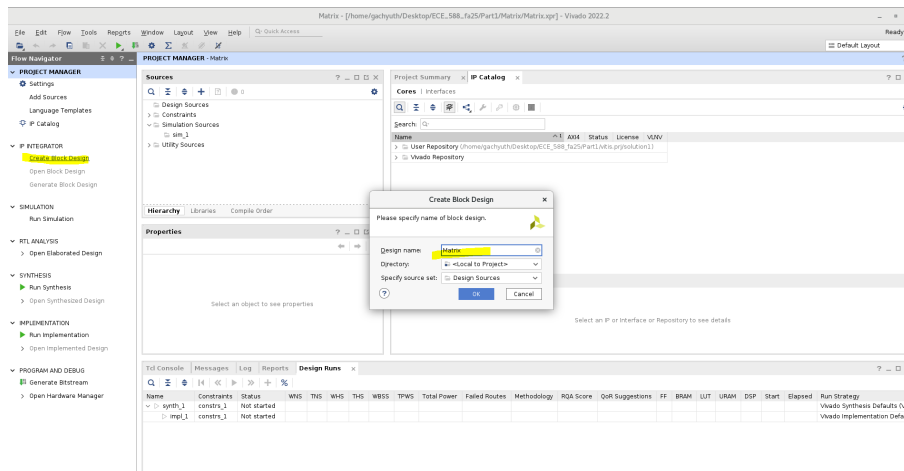


Figure 14: Creating a block design in Vivado

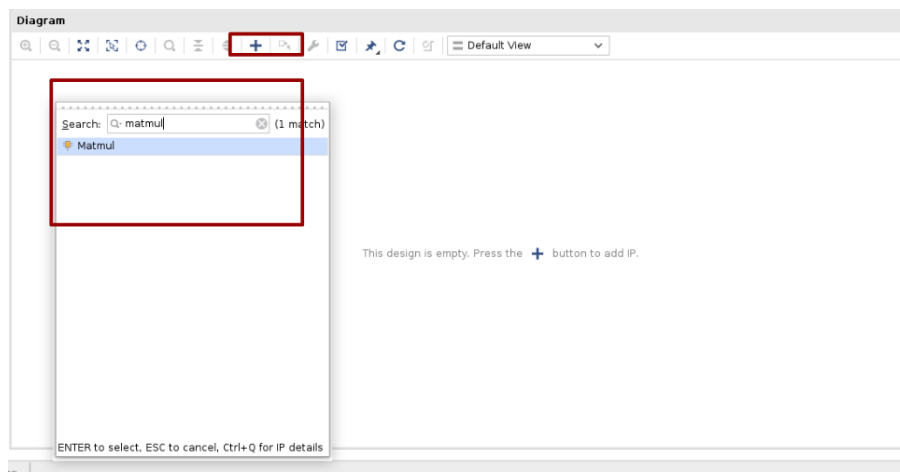


Figure 15: Adding the matrix multiplication accelerator IP

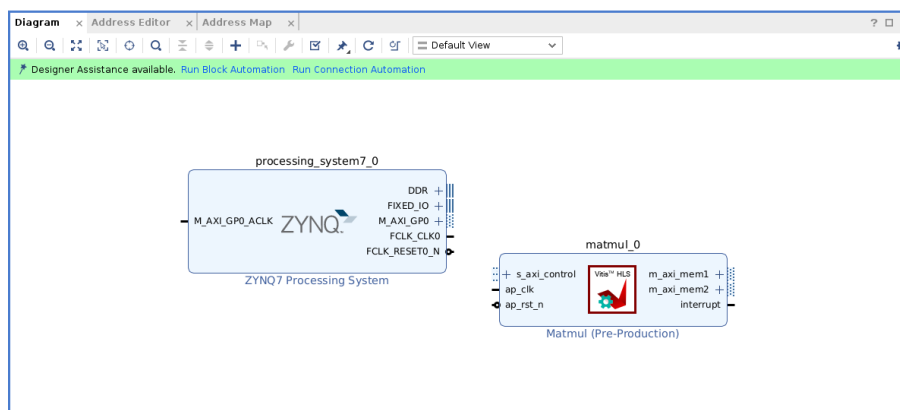


Figure 16: Design after adding Zynq PS and MatMul IP

5.8 Connecting Interfaces and Automating Design

Next, connect all components and automate the block connections:

- Click on **Connection Automation**.

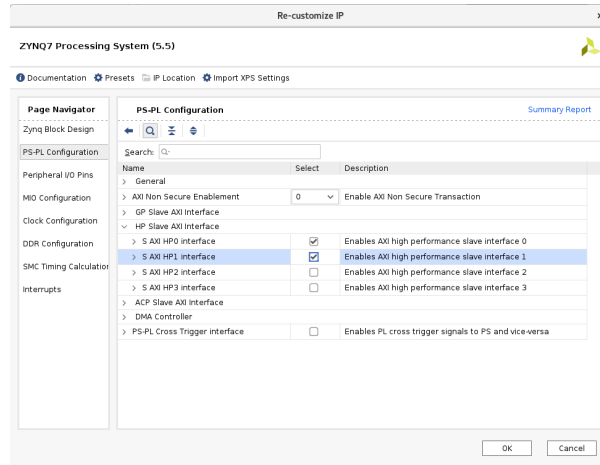


Figure 17: Enabling PS-PL interfaces

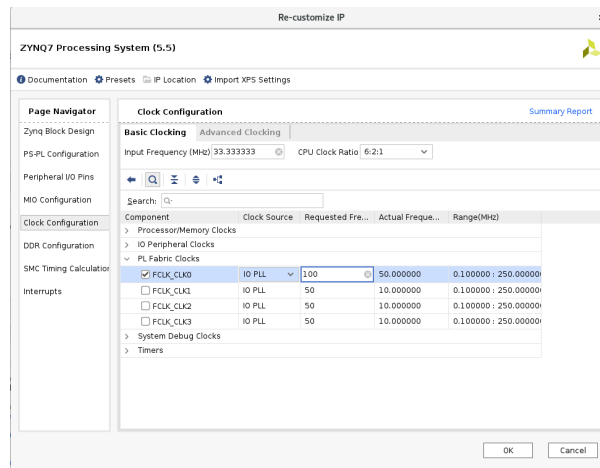


Figure 18: Configuring PL clock to 100 MHz

- Ensure that the **S AXI HP0** port is connected to the **M AXI HP1** port.
- Change **S AXI HP1** to **M AXI HP1** if prompted.
- Run **Block Automation**, select all suggested options, and uncheck **Board Preset**.
- Go to the **Address Editor** tab and click **Assign All**.

After these steps, your block design should resemble the one shown below, with all components properly connected.

6 Generating Bitstream and HWH

Once the block design is complete, the next step is to generate the hardware files required to program the FPGA.

1. Click **Validate Design** to check for errors or warnings.
2. Save the block design.

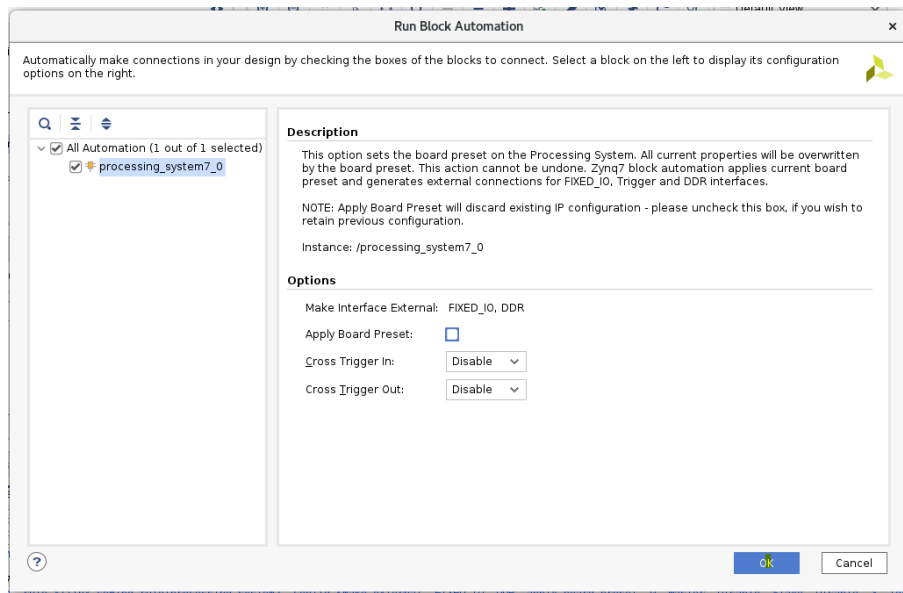


Figure 19: Running block automation

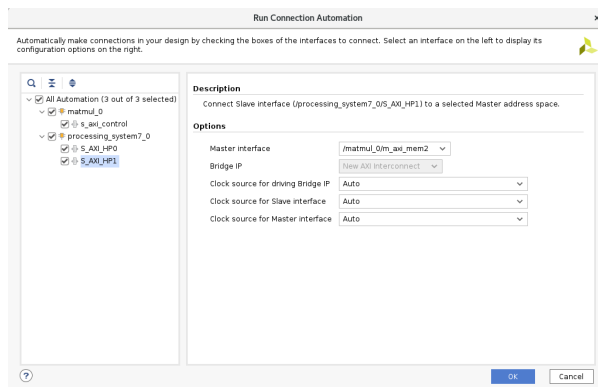


Figure 20: AXI port assignment during connection automation

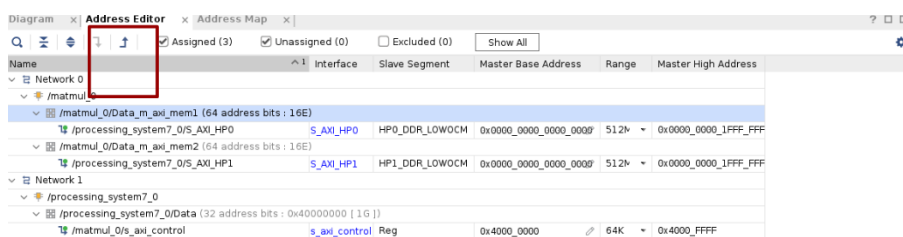


Figure 21: Assigning address spaces to peripherals

3. Switch to the **Sources** tab and right-click on **matrix.bd**.
4. Select **Create HDL Wrapper**. In the dialog box, choose **Let Vivado manage the wrapper automatically**.

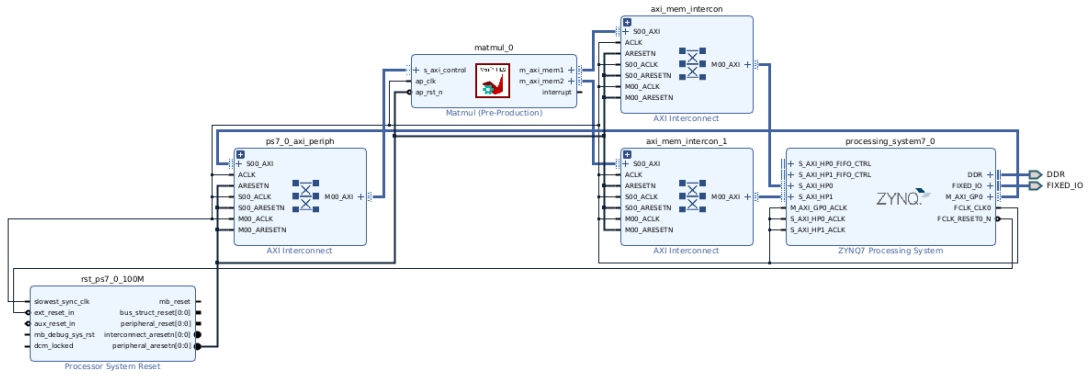


Figure 22: Final block design with Zynq PS, MatMul IP, and AXI interconnect

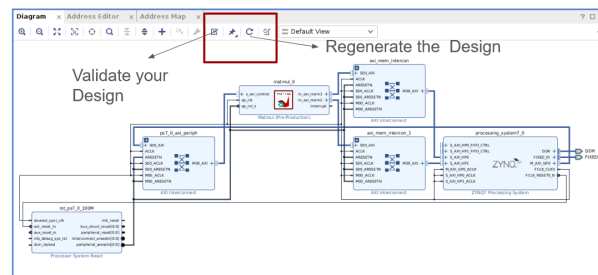


Figure 23: Validating and regenerating the block design

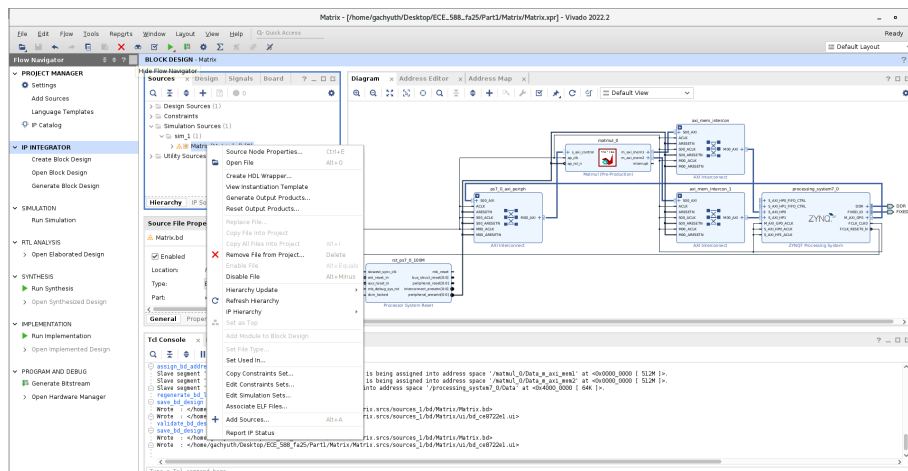


Figure 24: Creating the HDL wrapper for the block design

Once the HDL wrapper is generated:

1. Run **Synthesis**.
2. Run **Implementation**.
3. Finally, run **Generate Bitstream**. It takes some time be patient

This process produces two key files:

- **.bit** – the FPGA bitstream used to program the device.

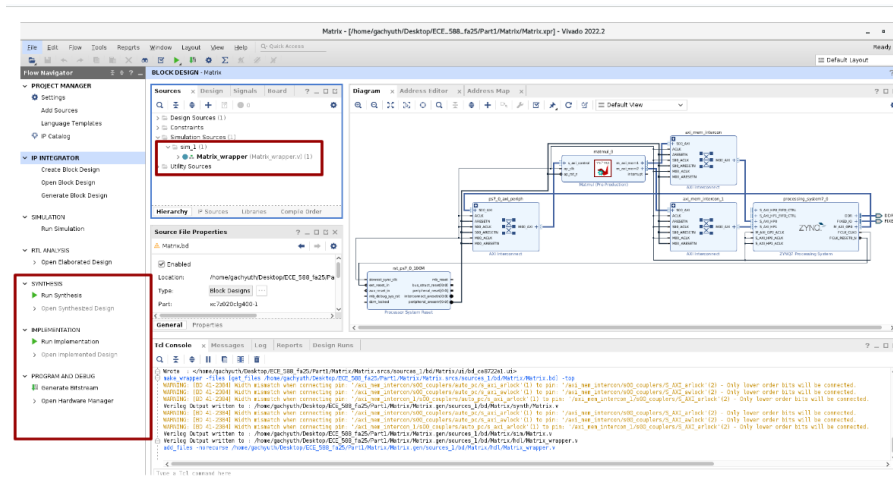


Figure 25: Generating the bitstream

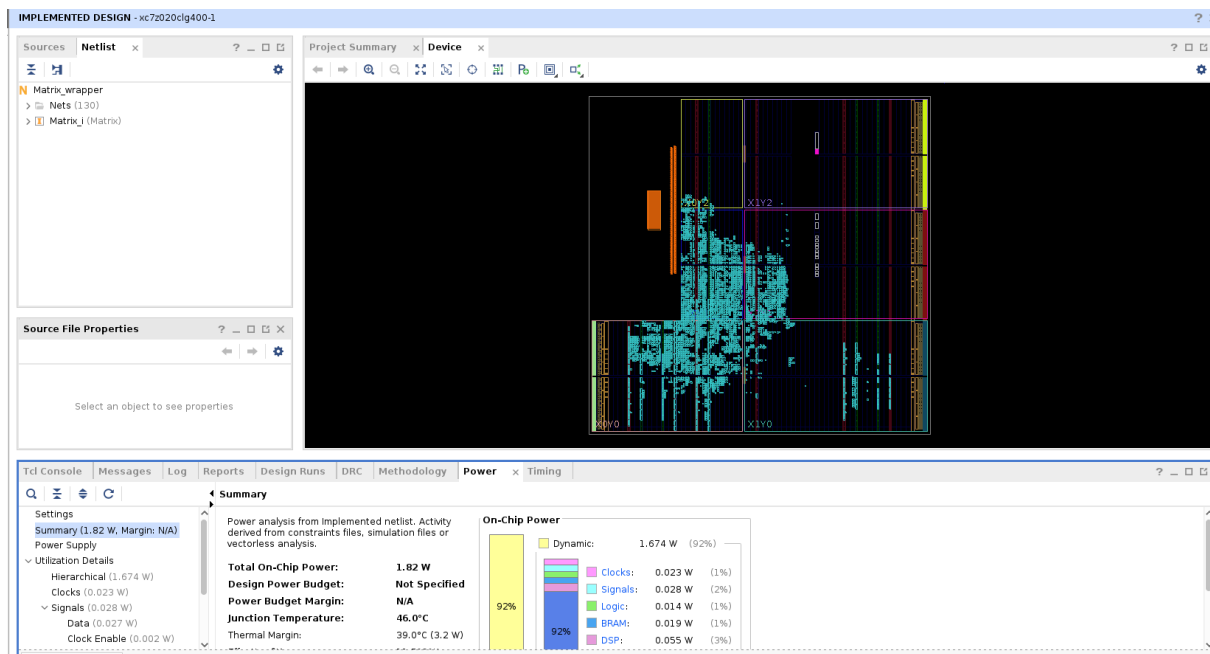


Figure 26: Implented Design after running Bitstream

- **.hwh** – a hardware handoff file containing metadata such as register maps and interface information.

7 Understanding Overlays

An **overlay** is a pre-built FPGA hardware design consisting of a **.bit** file and a **.hwh** file, which can be dynamically loaded and controlled from software without re-synthesizing the hardware design. Overlays allow developers to reconfigure the FPGA fabric and interact with hardware accelerators directly from software environments like Python or Jupyter Notebooks.

- The **.bit** file configures the FPGA fabric with your custom hardware accelerator.

- The `.hwh` file provides metadata (e.g., register maps, AXI interface descriptions) used by frameworks like PYNQ to control and interface with the accelerator.

Once an overlay is generated, it can be loaded into the FPGA directly from Python, enabling hardware execution without rerunning synthesis or implementation tools.

Tip: Overlays enable rapid hardware-software co-design. Once generated, they can be reused across different projects and applications without re-running synthesis and implementation flows.

Think of an overlay as a reusable **hardware library** that software can invoke just like a function.

7.1 Deploying on PYNQ-Z2

8 Login and Setup

Your group login credentials for the PYNQ-Z2 board are provided in an Excel sheet. Use the IP address to access the board through a browser. The default username and password are both:

Username: `xilinx`

Password: `xilinx`



Figure 27: Login screen for the PYNQ-Z2 board

9 Accessing the Terminal

After logging in, open a new terminal from the web interface:

Navigate to the working directory:

```
cd /home/xilinx
ls
```

10 Copying the Overlay Files

Locate the paths to your generated `.bit` and `.hwh` files on your server:

Copy both files to the PYNQ board, typically into:

```
/home/xilinx/overlays/matmul/
```



Figure 28: Opening the terminal on the PYNQ board

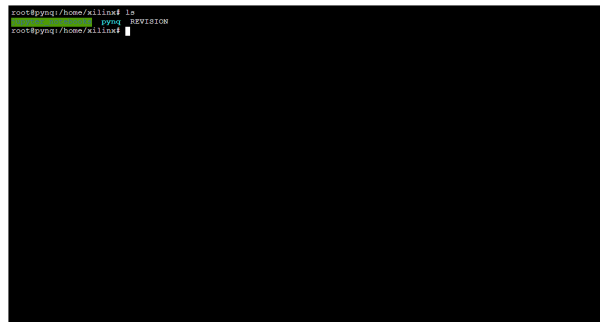


Figure 29: Listing files in the terminal

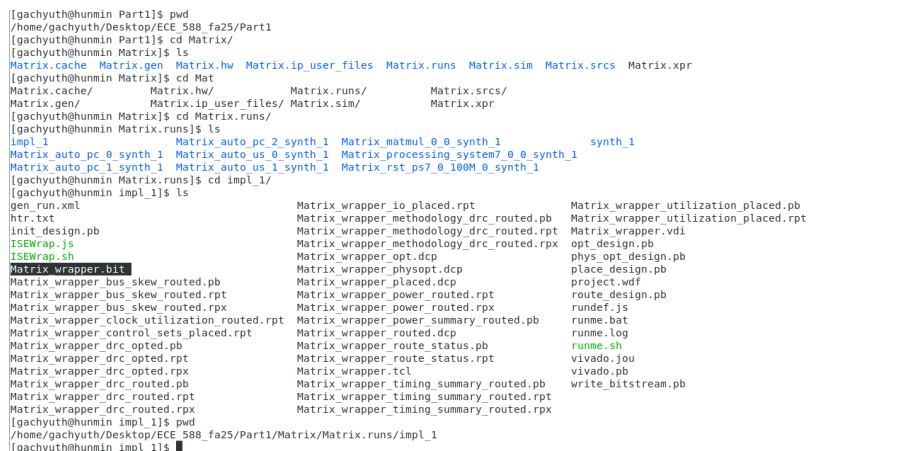


Figure 30: Path to the .bit file

11 Running the Overlay

Launch a Jupyter Notebook by selecting ****New → Python 3****:

Copy the provided code from `part1.ipynb` (on Colab or locally) into the notebook and run all cells.

11.1 Overlay Deployment Steps

1. Copy `.bit` and `.hwh` to `/home/xilinx/overlays/matmul/`.
2. Ensure filenames match: `matmul.bit` and `matmul.hwh`.

```

gachyuth@hunmin:hw_handoff
File Edit View Search Terminal Help
Matrix_wrapper_bus_skew_routed.rpt      Matrix_wrapper_power_routed.rpt      route_design.pb
Matrix_wrapper_bus_skew_routed.rpx      Matrix_wrapper_power_routed.rpx      rundef.js
Matrix_wrapper_clock_utilization_routed.rpt Matrix_wrapper_power_summary_routed.pb runme.bat
Matrix_wrapper_control_sets_placed.rpt   Matrix_wrapper_routed.dcp             runme.log
Matrix_wrapper_drc_opted.pb              Matrix_wrapper_route_status.pb        runme.sh
Matrix_wrapper_drc_opted.rpt             Matrix_wrapper_route_status.rpt       vivado.jou
Matrix_wrapper_drc_opted.rpx             Matrix_wrapper_tcl                    vivado.pb
Matrix_wrapper_drc_routed.pb             Matrix_wrapper_timing_summary_routed.pb write_bitstream.pb
Matrix_wrapper_drc_routed.rpt            Matrix_wrapper_timing_summary_routed.rpt
Matrix_wrapper_drc_routed.rpx            Matrix_wrapper_timing_summary_routed.rpx
[gachyuth@hunmin impl_1]$ pwd
/home/gachyuth/Desktop/ECE_588_fa25/Part1/Matrix/Matrix.runs/impl_1
[gachyuth@hunmin impl_1]$ cd ..
[gachyuth@hunmin Matrix.runs]$ cd ..
[gachyuth@hunmin Matrix]$ ls
Matrix.cache Matrix.hw Matrix.ip_user_files Matrix.runs Matrix.sim Matrix.srcs Matrix.xpr
[gachyuth@hunmin Matrix]$ cd M
Matrix.cache/      Matrix.hw/      Matrix.runs/      Matrix.srcs/
Matrix.gen/        Matrix.ip_user_files/ Matrix.sim/        Matrix.xpr
[gachyuth@hunmin Matrix]$ cd Matrix.gen
[gachyuth@hunmin Matrix.gen]$ ls
sources_1
[gachyuth@hunmin Matrix.gen]$ cd sources_1/
[gachyuth@hunmin sources_1]$ ls
bd
[gachyuth@hunmin sources_1]$ cd bd
[gachyuth@hunmin bd]$ ls
Matrix
[gachyuth@hunmin bd]$ cd Matrix/
[gachyuth@hunmin Matrix]$ ls
hdl hw_handoff ip ipshare Matrix.bda Matrix.bxaml Matrix.ooc.xdc sim synth
[gachyuth@hunmin Matrix]$ cd hw_handoff/
[gachyuth@hunmin hw_handoff]$ ls
Matrix.hwh
[gachyuth@hunmin hw_handoff]$ pwd
/home/gachyuth/Desktop/ECE_588_fa25/Part1/Matrix/Matrix.gen/sources_1/bd/Matrix/hw_handoff
[gachyuth@hunmin hw_handoff]$

```

Figure 31: Path to the .hwh file

```

root@pynq:/home/xilinx# scp -r gachyuth@hunmin.ece.iit.edu:/home/gachyuth/Desktop/ECE_588_fa25/Part1/Matrix/Matrix.runs/impl_1/Matrix_wrapper.bit matmul.bit
The authenticity of host 'hunmin.ece.iit.edu (216.47.158.178)' can't be established.
ECDSA key fingerprint is SHA256:mL2wnzVkiWk3nJCDLD1W+9vzaoclg/sEB4X4Aeb5Pdo.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'hunmin.ece.iit.edu,216.47.158.178' (ECDSA) to the list of known hosts.
gachyuth@hunmin.ece.iit.edu's password:
Matrix.wrapper.bit                                100% 3951KB  7.5MB/s   00:00
root@pynq:/home/xilinx# scp -r gachyuth@hunmin.ece.iit.edu:/home/gachyuth/Desktop/ECE_588_fa25/Part1/Matrix/Matrix.gen/sources_1/bd/Matrix/hw_handoff/Matrix.hwh matrix.hwh
gachyuth@hunmin.ece.iit.edu's password:
Matrix.hwh                                         100% 310KB   6.1MB/s   00:00
root@pynq:/home/xilinx# ls
matmul.bit matrix.hwh pynq REVISION
root@pynq:/home/xilinx#

```

Figure 32: Copying the overlay files to the PYNQ board



Figure 33: Opening Jupyter Notebook on the board

3. Open a Jupyter Notebook on the board.
4. Load the overlay.
5. Verify functionality by comparing FPGA and CPU results.

12 Performance Measurement Workflow

Once the design is deployed:

```

Matrix_C_DRAM_1 = Register(Matrix_C_DRAM-write-only),
Matrix_C_DRAM_2 = Register(Matrix_C_DRAM-write-only)
}
Pointer fields present: ['Matrix_A_DRAM_1', 'Matrix_A_DRAM_2', 'Matrix_B_DRAM_1', 'Matrix_B_DRAM_2', 'Matrix_C_DRAM_1', 'Matrix_C_DRAM_2']
A.shape, B.shape, C.shape: (64, 64) (64, 64) (64, 64)
CPU (trunc16, 1 run) : 21624.33 ms
CPU NumPy (int32 acc) median over 5: 3.32 ms
FPGA times (ms): ['0.409', '0.506', '0.409', '0.394', '0.392']
FPGA median (ms): 0.409
Max |HW-ExactCPU| = 0
PASS

see Performance.py
CPU time (used for speedup): 3.323 ms
FPGA time (kernel only) : 0.409 ms
FPGA perf: 1.282 GFLOP/s
Speedup (CPU/FPGA) = 8.13x

A shape=(64, 64) (top-left 8x8)
[[-70 -83 97 55 100 31 -33 -12]
 [-56 -82 -47 11 50 78 -35 -88]
 [-2 -18 -42 -35 -62 82 -11 -26]
 [ 32 -52 -95 37 65 27 64 -72]
 [-95 15 90 27 -39 13 -24 11]
 [ 73 100 -10 33 -12 -18 94 -19]
 [-55 22 50 -28 -17 92 -55 -83]
 [ 71 45 -66 -39 -22 62 47 16]]

B shape=(64, 64) (top-left 8x8)
[[ -4  32 -100 -66 -31 -40  32  28]
 [-82 -48 -35  24  34  9 -74  98]
 [ 82 -52 49  2 -60 15 93  99]
 [-11 18 -65 20 88 -17 70  84]
 [ -4 -10 25 -90 -9 -42 40  6]
 [-73 -67 48 76 55 6 31  13]]

```

Figure 34: Inference Time CPUvs FPGA and Sample Matrix

1. Run matrix multiplication on the CPU and measure execution time.
2. Run the same computation on the FPGA accelerator.
3. Measure FPGA execution time (kernel and data transfer).
4. Compute speedup:

$$\text{Speedup} = \frac{\text{CPU execution time}}{\text{FPGA execution time}} \quad (1)$$

13 Performance Notes

- FPGA acceleration provides significant performance improvement for large matrix sizes due to hardware-level parallelism.
- For small matrices, CPU and FPGA performance may be similar due to data transfer overhead.
- Use AXI DMA for large data transfers to maximize throughput.

14 Experimental Results

15 Measured Performance

The following results were obtained for matrix multiplication of size 64×64 :

The maximum difference between FPGA and CPU results was:

$$\text{Max } |HW - CPU| = 0$$

This confirms that the FPGA implementation produces results identical to the CPU reference implementation.

Table 1: Performance results comparison

Metric	Value
CPU execution time (NumPy, median of 5 runs)	3.323 ms
FPGA execution time (kernel only)	0.409 ms
CPU performance	0.158 GFLOP/s
FPGA performance	1.282 GFLOP/s
Speedup (CPU / FPGA)	8.13×

16 Sample Matrix Data

The top-left 8×8 sections of the input matrices used are shown below:

16.1 Matrix A

```
[[-70 -83  97  55 100  31 -33 -12]
 [-56 -82 -47  11  50  78 -35 -88]
 [ -2 -18 -42 -35 -62  28 -11 -26]
 [ 32 -52 -95  37  65  27  64 -72]
 [-95  15  80  27 -39  13 -24  11]
 [ 73 100 -10  33 -12 -18  94 -19]
 [-55 -25  20 -18  27 -92 -55 -83]
 [ 71  45 -66 -39 -22  62  47  16]]
```

16.2 Matrix B

```
[[-4 -32 -100 -66 -31 -40  32 -28]
 [ 56  45  -8  20  16 -39 -74 -98]
 [ 18 -52 -49 -42  65  15  93  39]
 [ 32  30 -25 -42 -40 -17  70  44]
 [-65  18  63  15 -18 -42  40  -6]
 [ 34 -91 -10  47  46  61  21  58]]
```

17 Conclusion

This project demonstrated a complete FPGA acceleration workflow for matrix multiplication:

- Hardware design and verification using HLS.
- Resource and performance evaluation using `csynth.rpt`.
- IP packaging and system integration in Vivado.
- Overlay generation and deployment on the PYNQ-Z2 board.
- CPU vs. FPGA performance benchmarking and analysis.

The FPGA accelerator achieved a speedup of approximately $8.13\times$ compared to a CPU-based implementation while producing identical results. This highlights the advantages of FPGA-based acceleration for compute-intensive applications, enabling hardware-level parallelism and significant performance improvements while maintaining software programmability.

*Prepared by Achyuth Gundrapally, Teaching Assistant for ECE588 – IIT Chicago,
September 2025*